Islamic University of Technology

Department of Computer Science and Engineering(CSE)

B.Sc. in Software Engineering

SWE 4404: Software Project Lab II

Summer, 2024-25

Project Proposal

LazyCow

Team 8

Tamjidur Rahman	230042109
Shudipto Sarwar	230042142
Irfan Sadik	230042147
Mohammad Faiaj Jarif	230042152

Supervisor

Samnun Azfar

Junior Lecturer, CSE

Date: 7 April, 2026

Contents

Project Overview

LazyCow is a simple shortcut-creation tool for Windows that opens applications and tabs with a few clicks, instead of opening them one by one. It is designed to automate the repetitive process of opening multiple applications, browser tabs, files, and folders before starting a work session.

The system allows users to combine desktop applications, URLs, files, and folders into customizable shortcuts. These shortcuts can be triggered instantly using predefined hotkeys, saving time and reducing the effort required to set up a working environment.

LazyCow also introduces a community-driven shortcut library that lets users share useful workspace setups with others. Shared shortcuts are designed to be customizable so that different users can adapt them according to their own needs (for example, different classroom links).

Motivation behind the Project

Every day we sit down at the computer and do the same ritual:

- Open the code editor
- Open the browser
- Open the same 6–7 tabs
- Open a folder
- Visit a couple of websites we always use

None of these tasks is hard. They're just annoyingly repetitive.

That tiny setup routine quietly eats a few minutes and a bit of our energy before we even begin the real work. And when you do this every single day, it starts to feel unnecessary.

That's where the idea of LazyCow came from.

We wanted a tool for people like us who are *comfortably lazy but still care about efficiency*. A tool where you press one hotkey and your entire workspace opens up, ready to go.

User Requirement Analysis

Functional Requirements:

1. Users can create shortcuts consisting of:
    - Applications
    - URLs
    - Files & Folders
2. Users can assign hotkeys to shortcuts.
3. Users can save, edit, and delete shortcuts.
4. Users can share shortcuts in a community library.
5. Users can customize shared shortcuts (e.g., different classroom URLs).
6. The system detects invalid or broken shortcuts before execution.
7. User account system for managing shared shortcuts and cloud sync for different devices.

Non-functional Requirements:

1. Simple and user-friendly UI.
2. The application is suitable for integration with the Windows OS.
3. Fast Shortcut Execution.
4. Lightweight.
5. Secure storage of user data.
6. Reliability and error handling.

Flow Chart

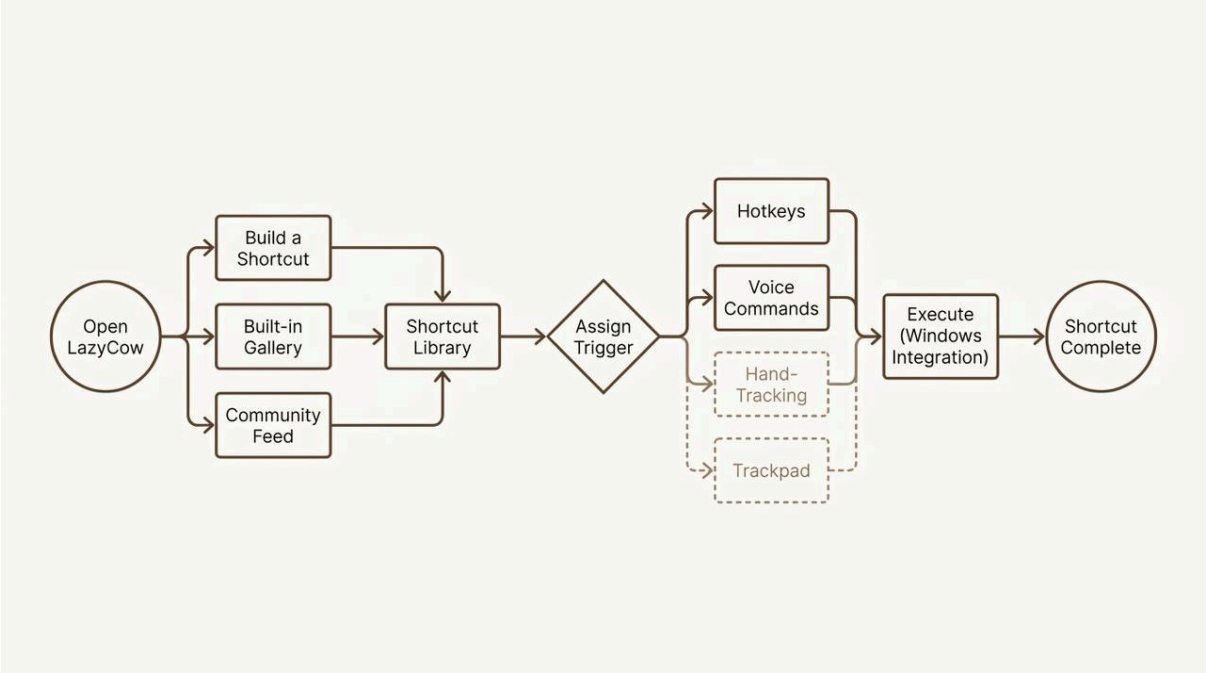

Figure 1: LazyCow Flowchart

Key Features

Main Features:

- Shortcut Builder for apps, URLs, files, folders
- Hotkey-based shortcut execution
- Community shortcut sharing
- Customizable shared shortcuts using placeholders
- Invalid shortcut detection
- Simple and clean UI
- Windows OS integration
- Account and guest mode support

Side and Potential Features:

- Voice command shortcuts
- Trackpad gesture-based shortcuts

Tools & Technologies

Languages

- **JavaScript** – Core programming language for application logic and automation

Frameworks / Libraries

- **Electron** – Desktop application framework for building Windows apps
- **Node.js** – Runtime environment used within Electron
- **Windows API (via Node packages)** – For OS interaction such as hotkeys, app launching, and window control

Database

- **SQLite** – Local storage for user-created shortcuts
- **Supabase** – Community shortcut sharing and user account management

Package Managers

- **npm** – Dependency and package management

Automation Utilities

- Libraries for browser automation and system control integrated through Node.js

Proposed Timeline

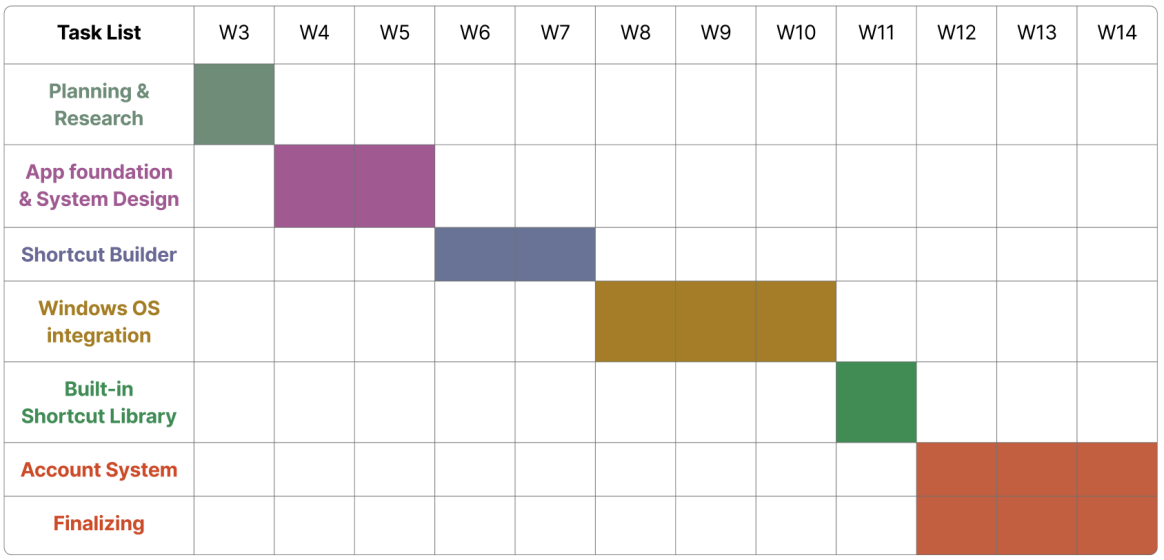

Figure 2: LazyCow Proposed Timeline

* The proposed timeline may be subject to changes depending on the progress.

Action taken based on the feedback

We carefully analyzed all feedback provided by the instructors during the proposal presentation and updated/ will update our project design accordingly.
The feedback we have received from the judge panel was:

1. Apply Tile-Based Window Transition (Inspired by Linux Tiling Window Managers)

   Feedback:
   After launching multiple applications, the screen may become cluttered. A tile-based window arrangement, like Linux tiling window managers, was suggested.

   Action Taken:
   We will implement automatic window arrangement after shortcut execution so that all opened applications are neatly organized on the screen in a tiled layout, providing an instantly ready and clean workspace.

2. Use a Headless Browser for Websites Requiring Login

   Feedback:
   Some websites included in shortcuts may require login or authentication.

   Action Taken:
   We will integrate a headless browser automation system. When a shortcut contains such a website, LazyCow will ask the user for their username and password and automatically log them in during shortcut execution.

3. **Problem of Different Users Having Different Classroom URLs**

   Feedback:
   If a user shares a shortcut containing a classroom or personalized

URL, it may not work for others because their class codes or links are different.

Action Taken:
We will introduce **dynamic placeholders** in shared shortcuts.

When another user downloads the shortcut, LazyCow will prompt them to enter their own class code. This makes shared shortcuts reusable and customizable.

4. Handling Websites that Require Authentication

Feedback:
Repeated manual login to websites reduces the benefit of automation.

Action Taken:
Shortcuts can be marked as **Authenticated Sites**. LazyCow will securely ask for credentials and reuse them during execution to ensure a seamless experience. (i.e., Headless Browser implementation)

5. Suggestion to Avoid SQLite for Local User Data

Feedback:
Using SQLite for simple local shortcut storage may be unnecessary overhead.

Action Taken:
This is not decided yet, but if agreed on, we will design LazyCow to store user shortcut data using local JSON-based storage within the device through Electron/Node.js file handling. This keeps the application lightweight and removes the need for database management for simple structured data.